

Multithreaded prime calculation with monitors

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly

Αναλογία με παραγωγό-καταναλωτή

Παρατηρούμε άμεσα ότι πρόκειται για πρόβλημα ενός παραγωγού και πολλών καταναλωτών με buffer μεγέθους ενός στοιχείου.

Τα race conditions που παρατηρούμε είναι ότι ο κάθε καταναλωτής πρέπει να λαμβάνει μία μοναδική τιμή για να επεξεργαστεί και να μη χάνονται ενδιάμεσες τιμές.

Για να αποφύγουμε τα race conditions χρησιμοποιούμε mutexes και condition variables υλοποιώντας monitors. Δηλαδή παραλλαγή της προηγούμενης λύσης με σηματοφόρους.

Λειτουργία main

Το main thread λειτουργεί ως παραγωγός. Διαβάζει τιμές από την είσοδο stdin και όσο δεν ανιχνεύσει EOF δίνει την τιμή που διαβάζει στον επόμενο διαθέσιμο worker, χρησιμοποιώντας σηματοφόρους για να αποφευχθούν τα race conditions.

```
while(scanf != EOF)
{
    lock(mutex);
    while(!empty)
        wait(condition);
    value = temp;
    empty = 0;
    signal(condition);
    unlock(mutex);
}
empty = 0;
on = 0;
signal(condition);
lock(mutex);
while(working) wait(condition);
unlock(mutex);
```

Λειτουργία του worker thread

To worker thread λειτουργεί ως καταναλωτής:

```
while(on)
{
    lock(mutex);
    while(empty)
        wait(condition);
    if(!on) break;
    temp = value;
    empty = 1;
    broadcast(condition);
    unlock(mutex);
    // process value
}
working--;
signal(condition);
unlock(mutex);
```

Γέφυρα

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly

Περιγραφή λύσης

Ο στόχος είναι να διέρχονται οχήματα έτσι ώστε:

- Να μην υπάρχει περίπτωση δύο οχήματα αντίθετης κατεύθυνσης να βρίσκονται πάνω στη γέφυρα,
- Να μην υπάρχει λιμοκτονία δηλαδή ένα αυτοκίνητο να μην μπορέσει να περάσει ποτέ τη γέφυρα
- Να μην υπάρχουν πάνω στη γέφυρα παραπάνω οχήματα από τη χωρητικότητά της.

Επιτυγχάνουμε αμοιβαίο αποκλεισμό με ένα mutex και κάποια condition variables.

Περιγραφή λύσης

Ένα αυτοκίνητο επιτρέπεται να περάσει τη γέφυρα όταν:

- Γέφυρα κενή και είναι η σειρά του
- Γέφυρα έχει αυτοκίνητα ίδιου τύπου (αλλά δεν είναι γεμάτη)

Ένα αυτοκίνητο, περνώντας τη γέφυρα ελέγχει την ουρά των απέναντι αυτοκινήτων και την ουρά της δικής του πλευράς και αναλόγως λαμβάνει αποφάσεις για το ποιόν θα ξυπνήσει (αν ξυπνήσει κάποιον).

Για να βεβαιώσουμε την «δικαιοσύνη» στη λύση μας αλλάζουμε τη σειρά κατεύθυνσης όταν περάσουν το πολύ N αυτοκίνητα μιας κατεύθυνσης, όπου N το μέγεθος της γέφυρας.

Λειτουργία car thread

Το car thread ελέγχει την κατεύθυνσή του και αναλόγως περνά ή όχι την γέφυρα. Αντίστοιχα με το `if(dir == left)` και εντελώς συμμετρικά υλοποιείται η περίπτωση `if(dir == right)`.

```
lock(mutex);
cars++;
if(dir == left)
{
    while(right cars on bridge || (left cars on bridge && left cars count == capacity) || turn == right)
    {
        wait(leftCanPass);
    }
    if( (left cars on bridge + right cars on bridge) == 0 && turn == left) ||
    (left cars on bridge && left cars count < capacity && turn == left) )
    {
        if (other left cars waiting && not reached capacity)
            signal(leftCanPass);
        unlock(mutex);
        // cross left to right
        lock(mutex);
        depending on queues and turn variable, signal(leftCanPass) or signal(rightCanPass);
        unlock(mutex);
        break;
    }
}
if(cars == 0) signal(mainLeave);
```


Λειτουργία main thread

Η main thread παράγει αυτοκίνητα. Δεν διαχειρίζεται η ίδια τα αυτοκίνητα. Όλη η επικοινωνία για το ποιος θα περάσει γίνεται μεταξύ των ίδιων των αυτοκινήτων.

```
while(1)
{
    if(scan == EOF) break;
    create_thread(car);
}
on = 0;
If(cars)
    wait(mainLeave);
```

Τρενάκι

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly

Λειτουργία προγράμματος

Το train thread προσπαθεί να γεμίσει όσα περισσότερα rides μπορεί, με βάση τους επιβάτες που περιμένουν στο queue του. Όταν ειδοποιήσουμε από την main ότι πρέπει να τερματίσει, το τρενάκι ειδοποιεί τους εναπομείναντες επιβάτες να τερματίσουν και στη συνέχεια το τελευταίο thread που τερματίζει (είτε είναι το τρενάκι είτε ένας επιβάτης) κάνει signal στη main να ξεμπλοκάρει και να τερματίσει και αυτή.

Το κάθε thread-επιβάτης περιμένει να επιβιβαστεί και όταν ο τελευταίος παρατηρήσει ότι το τρένο έχει γεμίσει ειδοποιεί το τρένο να ξεκινήσει το τρέχον ride.

Λύνουμε τα race conditions με χρήση condition variables και ενός mutex. Μέσω του mutex προστατεύουμε τις κρίσιμες μεταβλητές όπως το queue, το capacity και το num.

Λειτουργία main

Δίνουμε στο main thread έναν αριθμό επιβατών για να γεμίσει το queue. Όταν στείλουμε EOF η main ειδοποιεί το τρένο να τερματίσει και εκείνο με τη σειρά του ειδοποιεί τους εναπομείναντες επιβάτες να τερματίσουν.

```
while(1)
{
    if(scanf == EOF) break;
    create_thread(passenger);
}
lock(mutex);
on = 0;
signal(train);
wait(mainDone);
unlock(mutex);
```

Λειτουργία των passenger threads

Οι επιβάτες μπαίνουν στο τρενάκι ένας ένας και ο τελευταίος (εκείνος που θα ανιχνεύσει ότι το τρενάκι έχει γεμίσει, ξυπνάει το train thread για να ξεκινήσει το τρέχον ride.

```
lock(mutex);
queue++;
while(boarded < capacity)
{
    wait(passenger);
    if(!on) break;
}
if(!on)
{
    queue--;
    if(queue == 0)
        signal(mainDone);
    broadcast(passenger);
    unlock(mutex);
    return;
}
queue--;
boarded++;
if(boarded == capacity) signal(train);
else signal(passenger);
unlock(mutex);
```

Λειτουργία του train thread

Το τρενάκι περιμένει να γεμίσει και όταν η main το ειδοποιήσει να τερματίσει, λέει στους εναπομείναντες επιβάτες στο queue να τερματίσουν και αυτοί.

```
while(1)
{
    lock(mutex);
    while(on && boarded < capacity)
    {
        wait(train);
        if(!on) break;
    }
    if(!on)
    {
        if(queue != 0) signal(passenger);
        else signal(mainDone);
        unlock(mutex);
        break;
    }
    unlock(mutex);
    // sleep for train ride duration
    boarded = 0;
    signal(passenger);
}
```